

# Contents

---

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>1</b> |
| 1.1      | Prologue                                                | 1        |
| 1.2      | Automation is everywhere                                | 1        |
| 1.3      | Contributions                                           | 1        |
| 1.3.1    | Alternative type-class solver in Coq-Elpi               | 1        |
| 1.3.2    | Static determinacy checking for Elpi                    | 1        |
| <b>2</b> | <b>State of the Art</b>                                 | <b>3</b> |
| 2.1      | The <i>Rocq</i> proof assistant                         | 3        |
| 2.1.1    | The basis of the language                               | 3        |
| 2.1.2    | Elaboration                                             | 3        |
| 2.1.3    | Ad-hoc polymorphism: Type Classes                       | 3        |
| 2.2      | The <i>Elpi</i> language                                | 4        |
| 2.2.1    | $\lambda$ -terms representation                         | 4        |
| 2.2.2    | Hints about unification                                 | 4        |
| 2.2.3    | Controlling backtracking: the ! operator                | 4        |
| 2.2.4    | <i>CHR</i> : goal suspension                            | 4        |
| 2.3      | <i>Coq-Elpi</i> : Rocq and Elpi interleaving            | 4        |
| 2.3.1    | Concluding remarks                                      | 4        |
| <b>3</b> | <b>Alternative type-class solver in <i>Coq-Elpi</i></b> | <b>5</b> |
| 3.1      | Why typeclass resolution in <i>Elpi</i> ?               | 5        |
| 3.2      | From Rocq goals to <i>Elpi</i> proofs                   | 5        |
| 3.2.1    | Rule encoding                                           | 5        |
| 3.2.2    | Goal encoding                                           | 5        |
| 3.3      | Alternative search indexing                             | 5        |
| 3.3.1    | Discrimination trees                                    | 5        |
| 3.3.2    | Some ideas on tabling                                   | 5        |
| 3.4      | Helping higher-order unification                        | 6        |
| 3.4.1    | $\eta$ -equivalent terms                                | 6        |
| 3.4.2    | $\beta$ -equivalent terms                               | 6        |
| 3.5      | Experience report                                       | 6        |
| <b>4</b> | <b>Determinacy checking for Elpi</b>                    | <b>7</b> |
| 4.1      | Why determinacy?                                        | 7        |
| 4.2      | Implementation                                          | 7        |
| 4.2.1    | Rule checking                                           | 7        |
| 4.2.2    | Mutual exclusion                                        | 7        |
| 4.2.3    | Caveats                                                 | 7        |
| 4.2.4    | Experience report                                       | 7        |

|          |                                    |           |
|----------|------------------------------------|-----------|
| 4.3      | Formalization in Rocq . . . . .    | 7         |
| 4.3.1    | The language . . . . .             | 7         |
| 4.3.2    | The tree semantics . . . . .       | 8         |
| 4.3.3    | The stack semantics . . . . .      | 8         |
| 4.3.4    | Semantics equivalence . . . . .    | 8         |
| 4.3.5    | The main theorem . . . . .         | 8         |
| <b>5</b> | <b>Conclusion and Perspectives</b> | <b>9</b>  |
|          | <b>Bibliography</b>                | <b>11</b> |

# CHAPTER 1

---

## Introduction

### 1.1 Prologue

Automation everywhere in CS, we want to set up boilerplates to help our development.

Examples:

- Continuous integration
- Macros are everywhere
- Artificial intelligence

We delegate tasks to some tools.

expert system →

example plus frappant:

artificial intelligence: based on probability but not reproducible, not predicatable

several other examples in computer sciences:

...

### 1.2 Automation is everywhere

In the domain of formal proof automation is everywhere, and it is formally called elaboration ??

In formal proof, proofs are verbose and have lot of details: types, coercions, canonical structures, type-classes.

In theorem provers the user benefit of notations, implicit inference and tactics to ease his job.

Simply typed lambda calculus is an example of automation: an algo checks if ...

### 1.3 Contributions

In this manuscript we focus on an alternative type-class solver written in the elpi meta-language.

We also talk about a formalized tool...

#### 1.3.1 Alternative type-class solver in Coq-Elpi

#### 1.3.2 Static determinacy checking for Elpi



# CHAPTER 2

## State of the Art

At the very base of my PhD there are two softwares, the Rocq Proof assistant (formerly known as *Coq* [3]) and the *Elpi* [8].

### 2.1 The *Rocq* proof assistant

[3]

Theorem provers with HO logic programming languages: [10]

Mathematical components: [24]

Abella: [16]

SSReflect: [17]

Isabelle: [29]

#### 2.1.1 The basis of the language

#### 2.1.2 Elaboration

In fig. 2.1 (gently offered by Enrico), we have a skemtical representation of the elaboration process.

#### 2.1.3 Ad-hoc polymorphism: Type Classes

One of the most interesting fragment of elaboration is [36, 34]

Cite Haskell [20], in caml it does not exists: `+. vs +`

IRIS: [23]

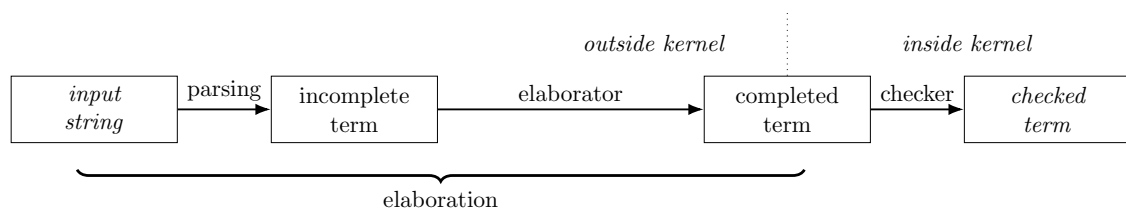


Figure 2.1: Elaboration skeme

## 2.2 The *Elpi* language

Elpi [8] is a dialect of  $\lambda$ prolog [26, 28]

Teyjus: [27]

### 2.2.1 $\lambda$ -terms representation

### 2.2.2 Hints about unification

Miller: [26]

Huet: [22]

### 2.2.3 Controlling backtracking: the ! operator

[35]

### 2.2.4 *CHR*: goal suspension

[15, 14]

CHR elpi: [19, section 4.3]

CHR Pfenning: [25]

## 2.3 *Coq-Elpi*: Rocq and Elpi interleaving

Hierarchy Builder: [6]

Trackt: [4]

Equality test [18]

### 2.3.1 Concluding remarks

Overall, Elpi is good for elaboration

# CHAPTER 3

---

## Alternative type-classes solver in *Coq-Elpi*

Typeclasses are an elegant way to implement ad-hoc polymorphism [37], so that, the same symbol can be reused in different context while keeping the same mathematical properties. From an implementation point of view, typeclasses are

### 3.1 Why typeclass resolution in *Elpi*?

[11]

Rule base automation!

### 3.2 From Rocq goals to *Elpi* proofs

#### 3.2.1 Rule encoding

The hooks

#### 3.2.2 Goal encoding

### 3.3 Alternative search indexing

#### 3.3.1 Discrimination trees

Discrimination trees: Pfenning notes \*

TODO: do some benchmarks

Discrimination trees: opt for lists and tree depths.

#### 3.3.2 Some ideas on tabling

[32, 30]

---

\*<https://www.cs.cmu.edu/~fp/courses/99-atp/lectures/lecture28.html>

## 3.4 Helping higher-order unification

[12]

?: [9]

Variadic functions are typechecked in coq [5]

### 3.4.1 $\eta$ -equivalent terms

### 3.4.2 $\beta$ -equivalent terms

[1]

Beta problems outside the pattern can come back in LL [27]

## 3.5 Experience report

IRIS: [23]

# CHAPTER 4

---

## Determinacy checking for Elpi

### 4.1 Why determinacy?

[7, 33, 21]

### 4.2 Implementation

#### 4.2.1 Rule checking

#### 4.2.2 Mutual exclusion

Using discrimination tree

#### 4.2.3 Caveats

For example, mutual exclusion is not perfect: over approximation of unification

Rule checking ignores flexible rule loading

#### 4.2.4 Experience report

### 4.3 Formalization in Rocq

Using ssreflect [17]

#### 4.3.1 The language

Input must come before outputs

Inputs must come before outputs, with Disc tree this is not mandatory, since unification is weaker.

EG:

```
pred g o:int, o:int.  
  
f 3 3 (g X X).  
f 3 4 (g 3 4).  
  
main:-  
  f X Y (g X Y).
```

In the code above, both rules can be used with the query, but, the discrimination tree, save the world

#### 4.3.2 The tree semantics

#### 4.3.3 The stack semantics

[\[31\]](#)

#### 4.3.4 Semantics equivalence

[\[2\]](#)

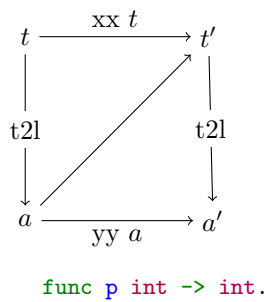
#### 4.3.5 The main theorem

# CHAPTER 5

---

## Conclusion and Perspectives

[13]





# Bibliography

---

- [1] Andreas Abel and Brigitte Pientka. Higher-order dynamic pattern unification for dependent types and records. In Luke Ong, editor, *Typed Lambda Calculi and Applications*, pages 10–26, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [2] Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- [3] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development. Coq’Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer Verlag, 2004.
- [4] Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
- [5] Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013.
- [6] Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34>, doi:10.4230/LIPIcs.FSCD.2020.34.
- [7] Saumya K. Debray and David S. Warren. Functional computations in logic programs. *ACM Trans. Program. Lang. Syst.*, 11(3):451–481, July 1989. doi:10.1145/65979.65984.
- [8] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/978-3-662-48899-7\_32.
- [9] Amy Felty. Encoding the calculus of constructions in a higher-order logic. In M. Vardi, editor, *lics93*, pages 233–244. IEEE, June 1993. doi:10.1109/LICS.1993.287584.
- [10] Amy Felty and Dale Miller. Specifying theorem provers in a higher-order logic programming language. In Ewing Lusk and Ross Overbeek, editors, *9th International Conference on Automated Deduction*, pages 61–80, Berlin, Heidelberg, 1988. Springer Berlin Heidelberg.
- [11] Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.

- [12] Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.
- [13] Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- [14] Thom Frühwirth. Constraint handling rules - what else? In Nick Bassiliades, Georg Gottlob, Fariba Sadri, Adrian Paschke, and Dumitru Roman, editors, *Rule Technologies: Foundations, Tools, and Applications*, pages 13–34, Cham, 2015. Springer International Publishing.
- [15] Thom Frühwirth. Theory and practice of constraint handling rules. *The Journal of Logic Programming*, 37(1):95–138, 1998. URL: <https://www.sciencedirect.com/science/article/pii/S0743106698100055>, doi:10.1016/S0743-1066(98)10005-5.
- [16] Andrew Gacek. The abella interactive theorem prover (system description). In *Proceedings of the 4th International Joint Conference on Automated Reasoning, IJCAR '08*, page 154–161, Berlin, Heidelberg, 2008. Springer-Verlag. doi:10.1007/978-3-540-71070-7\_13.
- [17] Georges Gonthier and Enrico Tassi. A language of patterns for subterm selection. In Lennart Beringer and Amy Felty, editors, *Interactive Theorem Proving*, pages 361–376, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [18] Benjamin Grégoire, Jean-Christophe Lécenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.
- [19] Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. *Mathematical Structures in Computer Science*, 29:1–26, 03 2019. doi:10.1017/S0960129518000427.
- [20] Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip L. Wadler. Type classes in haskell. *ACM Trans. Program. Lang. Syst.*, 18:109–138, March 1996. doi:10.1145/227699.227700.
- [21] Michael Hanus and Kai-Oliver Prott. Determinism Types for Functional Logic Programming. In *Proceedings of PPDP*, pages 47–59, 2025.
- [22] G.P. Huet. A unification algorithm for typed  $\lambda$ -calculus. *Theoretical Computer Science*, 1(1):27–57, 1975. URL: <https://www.sciencedirect.com/science/article/pii/0304397575900110>, doi:10.1016/0304-3975(75)90011-0.
- [23] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and DEREK DREYER. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi:10.1017/S0956796818000151.

- [24] Assia Mahboubi and Enrico Tassi. *Mathematical Components*. Zenodo, September 2022. doi:10.5281/zenodo.7118596.
- [25] Spiro Michaylov and Frank Pfenning. Higher-order logic programming as constraint logic programming. In *Position Papers for the First Workshop on Principles and Practice of Constraint Programming*, pages 221–229, Newport, Rhode Island, April 1993. Brown University.
- [26] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- [27] Gopalan Nadathur. The metalanguage  $\lambda$ prolog and its implementation. In Herbert Kuchen and Kazunori Ueda, editors, *Functional and Logic Programming*, pages 1–20, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [28] Gopalan Nadathur and Dale A. Miller. An overview of lambda-prolog. In *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, pages 810–827, Seattle, WA, USA, 1988. MIT Press.
- [29] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.
- [30] Brigitte Pientka. Tabling for higher-order logic programming. In Robert Nieuwenhuis, editor, *Automated Deduction – CADE-20*, pages 54–68, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [31] Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.
- [32] Daniel Selsam, Sebastian Ullrich, and Leonardo de Moura. Tabled typeclass resolution. *CoRR*, abs/2001.04301, 2020. URL: <https://arxiv.org/abs/2001.04301>, arXiv:2001.04301.
- [33] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming*, 29(1):17–64, 1996. High-Performance Implementations of Logic Programming Systems. URL: <https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684>, doi:10.1016/S0743-1066(96)00068-4.
- [34] Matthieu Sozeau and Nicolas Oury. First-class type classes. In *Proceedings of TPHOLs*, volume 5170 of *LNCS*, pages 278–293. Springer, 2008.
- [35] SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-Prolog PIDoc reference. URL: [https://www.swi-prolog.org/pldoc/doc\\_for?object=!/0](https://www.swi-prolog.org/pldoc/doc_for?object=!/0).

- [36] P. Wadler and S. Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '89, page 60–76, New York, NY, USA, 1989. Association for Computing Machinery. [doi:10.1145/75277.75283](https://doi.org/10.1145/75277.75283).
- [37] Philip Wadler. Theorems for free! In *Proceedings of FPCA*, FPCA '89, page 347–359. Association for Computing Machinery, 1989. [doi:10.1145/99370.99404](https://doi.org/10.1145/99370.99404).



# **Tritre francais**

Davide FISSORE